

Assignment No. 4

Aim : Create a Freestyle Jenkins job: Configure a job to print "Hello, Jenkins!" in the console output and view logs.

Objective: To understand and demonstrate the creation of a Freestyle Jenkins job by executing a basic build command and viewing the console output.

Theory

1. What is Jenkins?

Jenkins is an open-source tool that automates various stages of the software development lifecycle. It supports building, testing, and deploying applications across multiple environments, making it a critical component in CI/CD pipelines.

- **Automation:** Automates repetitive tasks such as builds, tests, and deployments.
- **Extensibility:** Over 1,800+ plugins to integrate with various tools like Docker, Kubernetes, Selenium, and Git.
- **Technology Agnostic:** Supports multiple programming languages and tech stacks, including Java, Python, JavaScript, and more.
- **Ease of Setup:** Provides a user-friendly web-based GUI for configuration.
- **Master-Slave Architecture:** Enables distributed builds for improved performance and scalability.

2. Types of jobs in Jenkins

1. Freestyle Project
 - a. This is the most basic type of Jenkins job.
 - b. It allows you to configure build steps manually using the Jenkins UI.
 - c. It is mainly used for simple build, test, or deployment tasks.
2. Pipeline
 - a. Pipeline jobs define the entire CI/CD process as a code using a Jenkinsfile.
 - b. It supports multiple stages like build, test, and deploy.
3. Multibranch Pipeline
 - a. This job automatically creates pipelines for each branch in a repository.
 - b. It is useful when working with feature branches in Git.
 - c. Each branch can have its own Jenkinsfile.
4. Maven Project
 - a. This job type is specifically designed for Maven-based Java projects.
 - b. Jenkins handles build and dependency management using Maven.
5. Folder
 - a. Folder jobs help organize multiple Jenkins jobs in a structured way
 - b. Useful for large projects with many pipelines.

6. External Job

- a. This is used when build processes run outside Jenkins.
- b. Jenkins only monitors and reports their status.

3. Jenkins Configuration

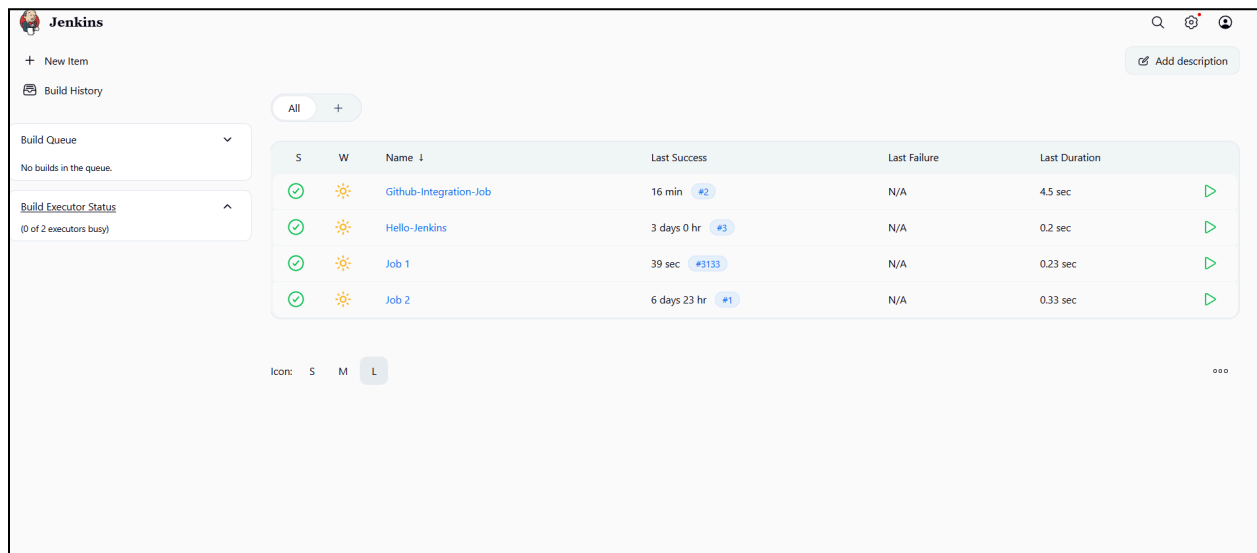
Jenkins configuration is the process of setting up Jenkins environment, tools and system setting for proper functioning. It includes:

- Global Tool configuration : Configure JDK, Git, Maven, etc.
- Manage Plugins : Install required plugins for extended features.
- Manage Nodes : Configure master and agent systems.
- Security Settings : User authentication and authorization
- System Configuration : Email notifications , environment variables, paths.

Procedure

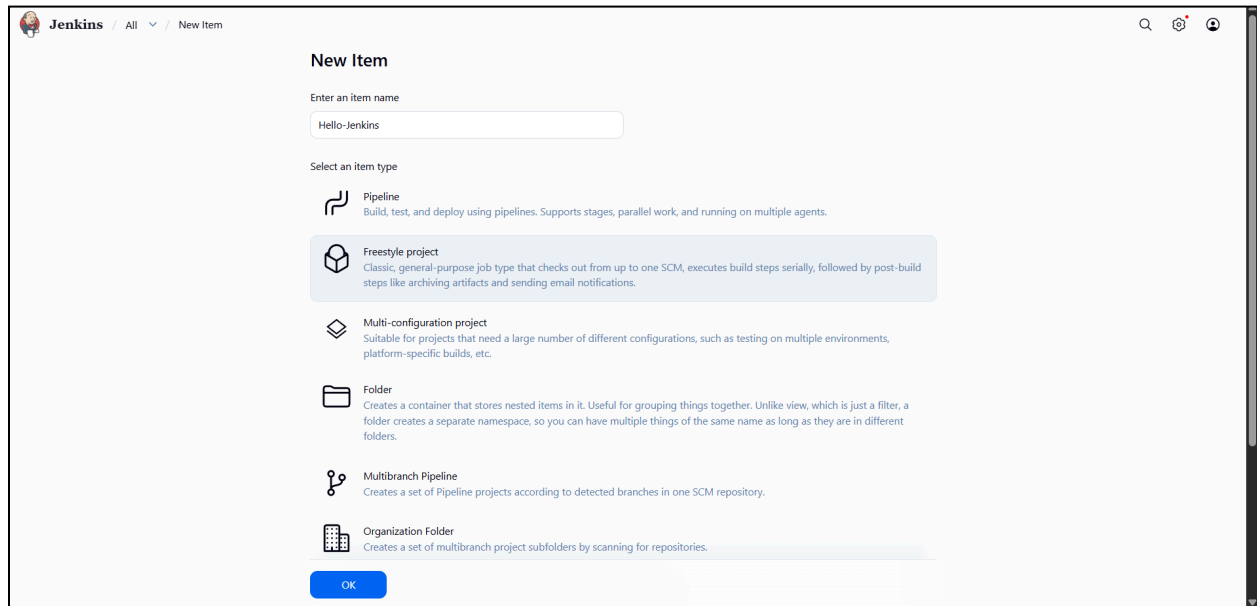
Step 1: Open Jenkins DashBoard

1. Open a web browser.
2. Enter Jenkins URL : <http://localhost:8080>
3. Login using Jenkins credentials.
4. Jenkins Dashboard will appear.



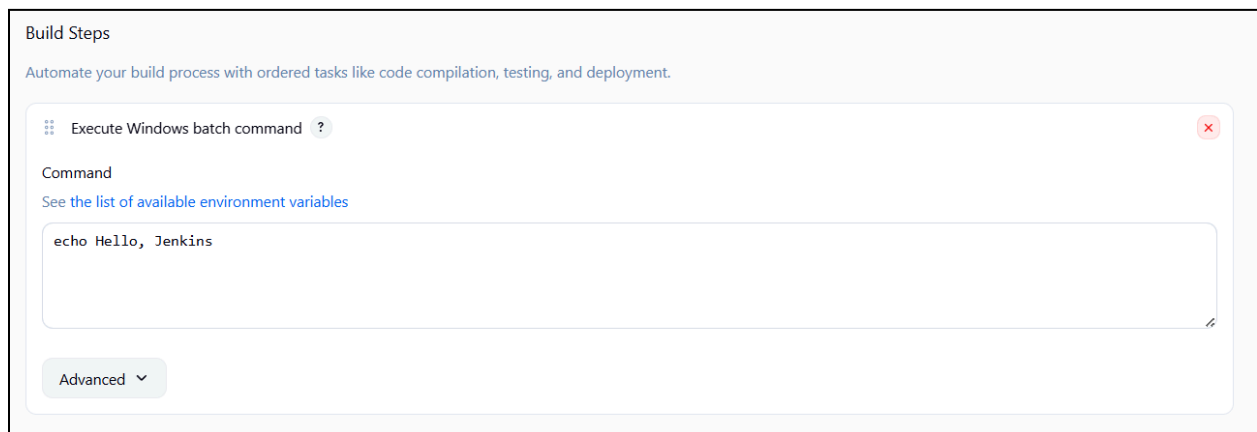
Step 2: Create a New Freestyle Job

1. Click “New Item” on the left panel
2. Enter Job Name. Example: “Hello Jenkins”
3. Select Freestyle Project.
4. Click OK.



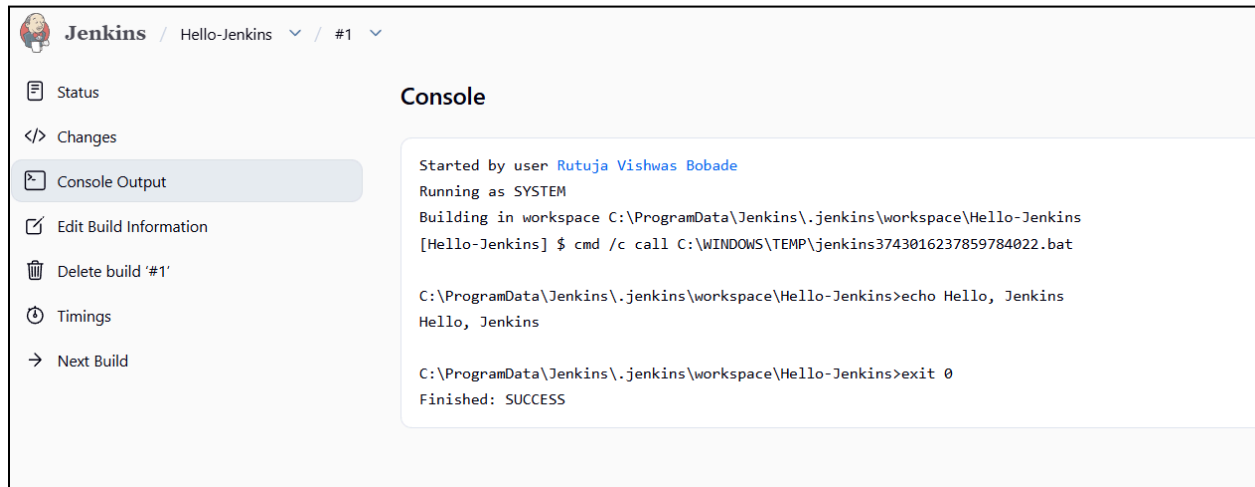
Step 3: Configure the Job

1. In general Section , we can add description.
2. Scroll down to Build Section.
3. Click on “**Add build step**” then select “**Execute Windows batch command**”.
4. Enter command.
For Windows : echo Hello, Jenkins
For Linux : echo “Hello, Jenkins”
5. Click Apply and Save.



5. Build the Job:

1. Click the Build number (i.e #1).
 2. Click Console Output.
 3. You can see the output as “Hello Jenkins”.
- Finished: SUCCESS



Result:

The Freestyle Jenkins job was successfully created and configured. The build executed the command and printed “Hello, Jenkins!” in the console output.

FAQ:

1. Why is Jenkins used in DevOps?

Jenkins is used in DevOps to automate the process of building, testing, and deploying applications, which helps teams deliver software faster and with fewer errors. It continuously integrates code changes from developers and runs automated pipelines, ensuring that problems are detected early and releases become more reliable. Jenkins acts as the automation engine of DevOps, connecting development and operations by enabling Continuous Integration and Continuous Delivery (CI/CD).

2. What is the purpose of Console Output in Jenkins?

The Console Output in Jenkins shows the detailed log of everything that happens during a job execution, such as code checkout, build steps, test results, and errors. It is mainly used for monitoring progress and debugging problems, helping developers understand whether a build succeeded or failed and why.

3. What is a build in Jenkins?

A build in Jenkins refers to a single execution of a Jenkins job or pipeline, where Jenkins performs tasks like fetching code, compiling it, running tests, and deploying the application. Each build is uniquely numbered and recorded, allowing developers to track results, view logs, and identify failures.

4. What is the role of plugins in Jenkins? Explain steps to install plugin.

Plugins extend the functionality of Jenkins by adding support for new tools, technologies, and features such as Git integration, Maven builds, Docker support, email notifications, and pipeline visualization. They allow Jenkins to connect with external systems and customize workflows, making Jenkins flexible and suitable for different DevOps needs.

Steps to Install a Plugin in Jenkins

1. Open Jenkins in your web browser.
2. Click on Manage Jenkins from the dashboard.
3. Select Manage Plugins.
4. Go to the Available tab and search for the required plugin.
5. Check the box next to the plugin name.
6. Click Install without restart (or Download now and install after restart).
7. Wait until installation completes, then restart Jenkins if prompted.